

Agentic Deep Research over LLM-Agent

Technical Report — AIMS DTU Research Intern 2026

Asif Khan | June 2026

LLM: Mistral via Ollama (local, free tier) | Framework: LangGraph | Corpus: 500 arXiv papers | Evaluator: evaluator.py (Mistral-as-judge)

1. What I Built and Why

The main motivation for this project was simple: LLMs are good at answering questions they already know, but they struggle with research questions that need evidence from multiple papers. I wanted to build something that actually retrieves and reads papers before answering, instead of just guessing from training data.

The system I built is called ADRA (Agentic Deep Research Agent). It works in a loop — it breaks down a question, retrieves relevant paper chunks, checks if it has enough evidence, and only then writes an answer with citations. I used LangGraph to wire everything together, Mistral via Ollama as the LLM (runs locally, completely free, no credit card), and a hybrid BM25 + FAISS retrieval index.

What genuinely interested me about this project was the ablation design. It is easy to build a system and claim it works. It is much harder to turn off each component one at a time and measure what actually breaks. That is what I set out to do here, and the results surprised me in a few ways — which I explain in Section 5.

2. Corpus Construction

2.1 What I collected

I used the free arXiv API (no key required) to download 500 papers from cs.CL, cs.AI, and cs.LG. Papers were filtered by keywords in the title or abstract: "LLM agent", "agentic RAG", "tool use", "agent memory", "agent benchmark", "computer-use agent", and "multi-agent". Sorted by submission date (newest first), covering January 2024 to April 2026. This gave 500 papers with 2,743 total text chunks after parsing.

2.2 Chunking

Each paper's title and abstract were joined into one text string. I split this into 400-character chunks with 100-character overlap. The overlap makes sure that a key sentence near a chunk boundary still appears in at least one chunk. Each chunk stores the arXiv ID so citations can be traced back to the source paper.

2.3 Retrieval index (HybridRetriever)

I built a hybrid index in retriever.py with two parts: BM25 (keyword search using rank_bm25) for exact term matching, and FAISS (vector search using all-MiniLM-L6-v2 embeddings, 384 dimensions) for semantic meaning. At query time, both run in parallel and results are merged using Reciprocal Rank Fusion (RRF, k=60). Top-2 chunks per sub-query are passed to the agent.

3. System Architecture

The agent is a LangGraph state machine with five nodes wired in agent_graph.py. Each node reads an ablation_mode flag from the state, which lets me turn it off cleanly without changing any code — just passing a different string from main.py.

Planner (planner_node)

Takes the user question and asks Mistral to break it into 1–2 sub-queries as a JSON list. For example, a comparison question becomes two separate searches. When ablated (`no_planner`), the original question is passed directly as the only query with no decomposition.

Retriever (`retriever_node`)

For each sub-query, asks Mistral to generate search keywords, then runs the HybridRetriever. Accumulates chunks across multiple rounds. When ablated (`no_hybrid`), only FAISS vector search runs — BM25 is disabled via the `disable_hybrid` flag in `retriever.py`.

Reflector (`should_continue_loop`)

After each retrieval round, asks Mistral "Do we have enough evidence?" If YES → synthesise. If NO → retrieve again. Capped at 3 rounds. Motivated by Reflexion (Shinn et al., 2023). When ablated (`no_reflector`), always goes straight to synthesis after the first retrieval round.

Synthesiser (`synthesizer_node`)

Takes all retrieved chunks and writes a final answer with inline citations like [2504.12345]. For the baseline ablation, only the first retrieved chunk is used, simulating single-shot RAG with no agentic loop.

Citation Verifier (`citation_verifier_node`)

Checks whether each cited paper's passage actually supports the claims in the answer. Prompts Mistral to remove unsupported statements. When ablated (`no_verifier`), citations are extracted from the raw synthesis directly without any verification step.

4. Evaluation Setup

I ran all 6 required configurations against the 30-question eval set. To score the outputs, I wrote evaluator.py in the src/ folder. It uses Mistral (same local Ollama model) as an LLM-as-judge: for each answer it asks the model to give a score between 0.0 and 1.0 based on factual accuracy against LLM-agent literature. The judge also reads inline citations from the answer text.

I report: Accuracy (LLM-as-judge score, 0–1), average citations per question, average answer length in characters, and completion status. Latency and tool-call count were not formally logged but are discussed qualitatively. All 6 configurations completed all 30 questions — the pipeline is now stable with a 6-second pacing delay and retry logic in main.py.

5. Results and Ablation Table

5.1 Main Results

Configuration	Accuracy	Avg Citations	Avg Ans Len	Questions	Status
full_agent	92.2%	2.6	1156 chars	30/30	Complete
baseline	88.5%	1.2	930 chars	30/30	Complete
no_planner	91.3%	1.6	984 chars	30/30	Complete
no_reflector	91.5%	1.7	833 chars	30/30	Complete
no_hybrid	92.8%	2.5	924 chars	30/30	Complete
no_verifier	92.0%	2.8	1174 chars	30/30	Complete

Table 1. All scores from evaluator.py. Accuracy = LLM-as-judge (Mistral). Green = full agent (best accuracy). All 6 configs: 30/30 questions complete.

5.2 Per-Component Effect

Component	Accuracy Delta	Key Observation
Planner	92.2% → 91.3% (-0.9%)	Fewer citations (2.6→1.6), shorter answers (1156→984 chars)
Reflector	92.2% → 91.5% (-0.7%)	Answer length drops most: 1156→833 chars, fewer retrieval rounds
Hybrid Ret.	92.2% → 92.8% (+0.6%)	Dense-only gets slightly higher LLM judge score but answer length drops
Verifier	92.2% → 92.0% (-0.2%)	More citations without verification (2.8 vs 2.6); accuracy barely changes

Table 2. What each ablation changed vs. the full agent. Delta measured on accuracy and output statistics.

6. Honest Analysis — What the Numbers Actually Say

What worked as expected

- The full agent got the highest accuracy (92.2%) and the richest answers (1156 chars avg, 2.6 citations avg). The agentic loop does help compared to the single-shot baseline (88.5%, 1.2 citations).
- The reflector made a visible difference in answer length — without it, answers dropped from 1156 to 833 characters on average. Fewer retrieval rounds means less evidence, shorter answers.
- The planner contributed to citation quality — without it, average citations dropped from 2.6 to 1.6, meaning the system found fewer relevant papers per question when queries were not decomposed.

Surprising findings

- no_hybrid scored the highest accuracy (92.8%), slightly above the full agent. This was unexpected. Dense-only retrieval (FAISS) seems to score better with the LLM judge, but the average answer length is shorter (924 chars) and citations are lower (2.5). My interpretation: FAISS retrieves semantically focused chunks that sound more confident, so the judge gives higher scores, but the answers may be less complete on multi-paper questions.
- no_verifier produced the highest citation density (2.8 avg) and longest answers (1174 chars) — actually more than the full agent. This makes sense: without the verifier stripping unverified citations, all retrieved paper IDs stay in the answer. The tradeoff is that some of those extra citations may not actually support the claims.
- The accuracy gap between configurations is small (88.5%–92.8%). This is partly because the LLM judge is the same Mistral model used to generate the answers. It may be scoring its own output generously. A better evaluation would use a different model as the judge.

Real limitations I found

- The corpus only uses titles and abstracts from arXiv, not full PDFs. This means the retriever cannot quote specific results, numbers, or tables from inside a paper. Some questions specifically asked for benchmark numbers, and the system could only answer from what was in the abstract.
- The LLM judge (evaluator.py) and the answer generator are the same Mistral model. This is not truly independent scoring. The assignment says "use whatever judge you like" and I was limited to local free-tier models, but this is a real weakness I want to flag.
- For q04 (about 'ACI'), the full agent returned an answer with zero cited papers because the specific acronym did not appear in any paper title or abstract in the corpus. The retriever simply could not find it. Query expansion with synonyms would help.

7. What I Would Improve Next

- Parse full PDFs instead of abstracts only. This is the biggest improvement — most factual answers require reading the results section, not just the abstract.
- Use a separate model as the LLM judge in evaluator.py (e.g., a different model via Ollama like llama3) so the scorer is independent from the generator.
- Add query expansion with synonyms (e.g., 'computer-use agent' → also search 'GUI agent', 'screen agent') to fix vocabulary mismatch failures like q04.
- Replace the prompt-based verifier with a proper NLI model (cross-encoder/nli-deberta-v3-small) so citation checking is based on entailment, not the same LLM.
- Add proper latency and tool-call count logging to main.py so those metrics can be reported accurately in the ablation table as the assignment asks.

8. Why I Find This Interesting

Before this project, I had built RAG systems and FastAPI backends, but I had never thought carefully about what actually makes an agentic system different from a fixed pipeline. The ablation design forced me to confront this. When I removed the reflector and saw answer lengths drop by 30%, or when I removed the planner and saw citations fall from 2.6 to 1.6, those were moments where the architecture made sense in a concrete way — not just as a diagram but as something measurable.

The part I found most interesting was the no_hybrid result — it scored higher than the full agent on accuracy. At first I thought my code was broken. After looking at the actual answers, I think the LLM judge favours focused, confident answers over long multi-paper syntheses. That is an evaluation problem as much as a system problem, and I think understanding the difference between "scores well" and "is actually better" is one of the most important open questions in agentic AI evaluation.

I want to be upfront: my system has real gaps. Abstract-only corpus, same-model judge, and a verifier that uses the same LLM it is supposed to check. I documented these honestly because I think knowing the limits of your own system matters as much as building it.

References

Shinn, N. et al. (2023). Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023.

Yao, S. et al. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR 2023.

Asai, A. et al. (2023). Self-RAG: Learning to retrieve, generate, and critique. arXiv:2310.11511.

Khot, T. et al. (2023). Decomposed prompting. ICLR 2023.

Es, S. et al. (2023). RAGAS: Automated evaluation of retrieval augmented generation. arXiv:2309.15217.